

German Language Domain — HTML Interface Checklist v1.0

Date: 2026-05-19

Build target: June 1, 2026

Plan: docs/GERMAN_HTML_BUILD_PLAN_v1.0.md

Spec: docs/GERMAN_HTML_SPEC_v1.0.md

One checkbox per discrete action. Complete in order. Do not skip Step 0.

Step 0 — Baseline Gate

- Run `_NewDomains/language-german/run_tests.py` — confirm all 49 tests pass
 - Create feature branch `feat/german-domain-extraction`
 - Commit baseline confirmation: `test: confirm 49-test baseline before extraction`
-

Step 1 — Group A: Data types and pure utilities

- Identify all constants and type definitions with no Telegram/async imports
 - Extract verb conjugation tables to `german_domain.py`
 - Extract phrase scoring logic (fuzzy match thresholds, promotion thresholds)
 - Extract scaffold phrase bank structure
 - Extract session JSON schema definitions
 - Extract persona file parsing utilities
 - Run all 49 tests — confirm green
 - Dry-run smoke check: run bot with `--dry-run`, confirm "next german session" responds correctly
 - Commit: `refactor(domain): extract Group A — [what]`
-

Step 2 — Group B: Domain logic without async dependencies

- Extract `_resolve_verb()` core logic (verb lookup + fallback)
- Extract `_resolve_phrases()` core logic (phrase selection + filtering)
- Extract scaffold assembly logic
- Extract persona selection logic with exact filename match before slug derivation (OpenClaw note #5)
- Extract session header assembly — verify compatibility with `parse_transcript.py` (OpenClaw note #1)
- Add Duration normalization helper — "number only" enforcement for BUG-014 (OpenClaw note #2)
- Add `--dry-run` guard: validation and reads only, no side effects (OpenClaw note #6)
- Add module docstring: both bots route to same endpoint (OpenClaw note #3)
- Add carry-forward output: error counts and vocabulary on separate lines (OpenClaw note #4)
- Run all 49 tests — confirm green
- Dry-run smoke check: run bot with `--dry-run`, confirm "next german session" responds correctly

- Commit: refactor(domain): extract Group B – [what]
-

Step 3 — Group C: Async operations with callback pattern

- Extract `_resolve_verb` with `progress_cb: callable | None = None` parameter
 - Extract `_resolve_phrases` with callback pattern
 - Extract `_resolve_drill_verb` with callback pattern
 - Audit `telegram_bot.py` for any other functions with mid-execution `reply_text` sends — extract each
 - Add test for each Group C function called with `progress_cb=None` — confirm no raise
 - Run all 49 tests — confirm green
 - Dry-run smoke check: run bot with `--dry-run`, confirm "next german session" responds correctly
 - Commit: refactor(domain): extract Group C – [what] with callback pattern
-

Step 4 — Group D: Wire adapters

- Update `telegram_bot.py` to import from `german_domain`
 - Verify `telegram_bot.py` contains: message routing, Whisper voice transcription, Telegram formatting, callback delivery — and nothing else
 - Run all 49 tests — confirm green
 - Dry-run smoke check: run bot with `--dry-run`, confirm "next german session" responds correctly
 - Smoke-test bot on staging (basic German drill and phrase commands)
 - Commit: refactor: wire telegram adapter to `german_domain`
 - Merge `feat/german-domain-extraction` to `main`
-

Step 5 — Flask server scaffold

- Create `html_server.py` with Flask app and basic routing
 - Set up static file serving from `static/`
 - Create six-tab navigation skeleton: Lesen, Schreiben, Üben, Bibliothek, Admin, Archiv
 - Add typography imports: Playfair Display + DM Sans + IBM Plex Mono
 - Define CSS custom properties: colour palette (charcoal, warm gray + one accent — terracotta OR deep teal)
 - Verify shell loads in browser with correct tab structure
 - Create feature branch `feat/german-html-interface`
 - Commit: feat(html): Flask scaffold with six-tab skeleton
-

Step 6 — Lesen tab

- Wire RSS fetcher to 4–5 seed sources (Heute, Spiegel Panorama, Kicker, ORF, Wiener Zeitung)
- Build item display: German headline, 2–3 paragraphs, source + date

- Add three action buttons per item: ■ ■ ■
 - Add pen icon → Schreiben handoff (passes headline + first paragraph as context)
 - Store thumbs feedback
 - Daily refresh: items persist until dismissed or next refresh
 - Confirm 3–5 items displayed (not more)
 - Commit: feat(html): Lesen tab – feed display and thumbs feedback
-

Step 7 — Schreiben tab (core)

- Freeform text input field — no forced prompt
 - Add rotating placeholder text: *"Was hast du heute gesehen?"* and similar Vienna scenes (≥5 variants)
 - Wire text submission to LLM correction via `german_domain`
 - Build correction display: original + corrected + notes (gender/case/word order)
 - Wire Lesen → Schreiben context pre-fill (article headline + first paragraph)
 - Add one-click phrase capture from correction view to phrase library
 - Show last 10 writing sessions (date, first line, error count, phrases captured)
 - Commit: feat(html): Schreiben tab – correction loop and Lesen handoff
-

Step 8 — Voice entry

- Add microphone button to Schreiben text field
 - Wire Web Speech API (Chrome macOS primary)
 - Test on target MacBook — confirm reliability
 - If unreliable: add Flask endpoint `POST /api/transcribe` (Whisper fallback)
 - Document Whisper fallback status (used / documented-but-not-wired) in code comment
 - Commit: feat(html): voice entry via Web Speech API
-

Step 9 — Bibliothek tab

- Build phrase browser: paginated, filterable by status / scene / verb_hint / date added
 - Implement inline editing (German/English pairs, verb_hint, status)
 - Create `lessons.json` data structure (separate file alongside `phrasebook.json`)
 - Build lesson grouping UI: create lesson, assign phrases, mark active/backlog/archived
 - Build drill pool visibility: verb card grid with phrase count + last drilled date
 - Click verb card → see all its phrases
 - Commit: feat(html): Bibliothek tab – phrase browser, lessons, drill pool
-

Step 10 — Üben tab

- Build session launcher: persona + scenario selector

- Add scaffold phrase preview before session start
 - Add carry-forward from last session display
 - Build transcript paste flow (alternative to Telegram paste)
 - Wire transcript submission to `reviewer.py` pipeline
 - Build session results view: error summary by type, scaffold deployment, Anki cards, carry-forward
 - Implement persona context/memory: rolling 3-session summary injected at session start
 - Test prompt size with Georg + Maria at max assembled size — flag if over 4096 chars (Telegram limit)
 - If over limit: mark persona memory as HTML-only for v1.0
 - Commit: `feat(html): Üben tab – session launcher and results view`
-

Step 11 — Admin tab

- Build persona editor form: name, role, location, register, personality notes, scenarios, scaffold phrases
 - Add prompt preview: show assembled persona prompt before saving
 - Implement persona create (writes new `.txt` file to `prompts/`)
 - Implement persona fork (versioned variant, new persona ID)
 - Implement persona edit (updates existing `.txt` file)
 - Commit: `feat(html): Admin tab – persona editor`
-

Step 12 — Look and feel pass

- Apply full typography: Playfair Display headings, DM Sans body, IBM Plex Mono for corrections/transcripts
 - Apply colour palette — warm dark neutrals + chosen accent (terracotta or deep teal)
 - Add subtle texture: very light grain/noise overlay on backgrounds
 - Verify correction display tone: warm, clear, not clinical
 - Mobile layout: single-column below 768px
 - Tab rail collapses to bottom nav on mobile
 - Touch-friendly tap targets throughout
 - Test on MacBook (Chrome) and iPhone (Safari) — basic navigation and Schreiben
 - Commit: `feat(html): look and feel pass – typography, colour, mobile layout`
-

Step 13 — Final gate before June 1

- All 49 tests pass on `main` (post-refactor)
- Lesen tab: feed loads, thumbs work, pen icon handoff works
- Schreiben tab: correction loop works, voice works (or fallback documented)
- Bibliothek tab: phrase browser loads, inline edit works, lessons work
- Üben tab: session launcher works, transcript paste works, results display works
- Admin tab: persona create and fork work
- Mobile layout: single-column navigation confirmed on iPhone
- Smoke-test Telegram bot — refactor must not have broken any existing commands

- Merge feat/german-html-interface to main
-

Step 14 — Post-merge (OpenClaw handles)

- Update PLAN.md with HTML interface milestone status
 - Update CHECKLIST.md — check off refactor and HTML build items
 - Reference GERMAN_HTML_BUILD_PLAN_v1.0.md in German domain roadmap
 - Update GERMAN.md status: v0.9 beta → v1.0 in progress
 - Update OPERATIONS_ROADMAP.md: add HTML interface as v1.0 milestone
 - File GitHub issue: feat: German domain HTML interface v1.0 – design approved
 - Labels: enhancement, language-domain, v1.0-milestone
 - Milestone: June 1, 2026
-

German Language Domain — HTML Interface Checklist v1.0

2026-05-19 — based on GERMAN_HTML_BUILD_PLAN_v1.0.md